

A young man with dark, curly hair and a goatee is smiling while wearing large black headphones. He is sitting and using a laptop. The background is a vibrant, out-of-focus outdoor space with graffiti on a wall and some greenery.

Arv med interfaces

Først en lille opsummering...

Komposition - **has-a** ... *en bankkonto "har en" ejer*

```
class BankAccount {  
    private Owner owner;  
    // ...  
  
    public BankAccount(double balance) {  
        this.owner = owner;  
        // ...  
    }  
  
    public Owner getOwner() {  
        return owner;  
    }  
  
    // ...  
}
```

Arv - **is-a** ... en opsparingskonto "er en" bankkonto

```
class BankAccount {  
    private double balance;  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    public void deposit(double amount) {  
        // ...  
    }  
  
    public void withdraw(double amount) {  
        // ...  
    }  
  
    // ...  
}
```

```
class SavingsAccount extends BankAccount {  
    private double interestRate;  
  
    public SavingsAccount(double balance, double interestRate) {  
        super(balance);  
        this.interestRate = interestRate;  
    }  
  
    void applyInterest() {  
        // ...  
    }  
  
    // ...  
}
```

```
class ChildrensAccount extends SavingsAccount {  
    public ChildrensAccount(double balance) {  
        super(balance, 0.01); // 1% rente  
    }  
    // ...  
}
```

... en børnekonto "er en" opsparingskonto, hvor du *ikke* kan hæve fra

Så må vi lave en klasse for børnekontoer:

```
class ChildrensAccount {  
    private Owner owner;  
    private double balance;  
    private double interestRate = 0.01; // 1% rente  
  
    public ChildrensAccount(double balance) {  
        this.owner = owner;  
        this.balance = balance;  
    }  
  
    public void deposit(double amount) {  
        // ...  
    }  
  
    void applyInterest() {  
        // ...  
    }  
  
    // Ingen withdraw-metode!  
}
```

Hmmm... hvorfor er det ikke optimalt?

... nu er **ChildrensAccount** ikke en **BankAccount** længere, og så kan vi ikke længere udnytte polymorfi:

```
ArrayList<BankAccount> accounts = new ArrayList<>();  
accounts.add(new BankAccount(owner1));  
accounts.add(new SavingsAccount(owner2, 0.05));  
accounts.add(new ChildrensAccount(owner3)); // FEJL!
```

Vi kan heller ikke

```
public void transfer(BankAccount from, BankAccount to, double amount) {  
    from.withdraw(amount);  
    to.deposit(amount);  
}  
  
transfer(childrensAccount, anotherAccount, 100); // FEJL!  
transfer(savingsAccount, childrensAccount, 100); // FEJL!
```

... fordi **ChildrensAccount** ikke er en **BankAccount** længere.



Kan vi så

```
public void transfer(BankAccount from, ChildrensAccount to, double amount) {  
    from.withdraw(amount);  
    to.deposit(amount);  
}
```

... tja, men så skal **transfer**-metoden kende til alle kontotyper, hvilket ikke er særlig fleksibelt.

Løsningen er **interfaces** !



# Interfaces

# Begrebsforvirringen

Interface betyder **grænseflade** på dansk. Vi bruger ofte ordet interface i forb. med brugergrænseflader, som nogle kalder brugerinterface.

Men her taler vi om noget andet: programmable interfaces, altså programmerbare grænseflader mellem klasser.

## API: Application Programming Interface

... det offentlige sæt af metoder (**public** metoder), en klasse tilbyder andre klasser, så de kan interagere med den.

Et interface i Java er en måde at beskrive et API på - hvad en klasse *kan gøre*.

Arv is-a - `class SavingsAccount extends BankAccount` vs.

vs.

Arv med interfaces can-do - `class SavingsAccount implements  
InterestBearing`



```
public interface InterestBearing {  
    void applyInterest();  
}
```

- ligner en klasse, `interface` i stedet for `public class`
- Metoder har ingen krop (ingen `{}`), kun signatur

```
public interface Depositable {  
    void deposit(double amount);  
}  
  
public interface Withdrawable {  
    void withdraw(double amount);  
}
```

Et interface er en kontrakt om, at den klasse, der implementerer interfacet, skal have de angivne metoder i sit API - dvs. metoderne skal være **public**.

En klasse *skal* implementere alle metoder i interfacet. Den kan implementere flere interfaces:

Dvs.

```
class SavingsAccount extends BankAccount implements InterestBearing, Depositable, Withdrawable {  
    // ...  
}
```

... betyder, at **SavingsAccount** skal have både en **applyInterest**-metode og en **deposit**-metode og en **withdraw**-metode.

men vores børneopsparing har *ikke* en **withdraw**-metode:

```
class ChildrensAccount extends BankAccount implements InterestBearing, Depositable
```

```
class ChildrensAccount extends BankAccount implements InterestBearing, Depositable {  
    // ...  
  
    // Implementing af interface InterestBearing  
    public void applyInterest() {  
        setBalance(getBalance() * (1 + interestRate));  
    }  
  
    // Implementing af interface Depositable  
    public void deposit(double amount) {  
        setBalance(getBalance() + amount);  
    }  
}
```

En checkkonto får ikke tilskrevet rente:

```
class CheckingAccount extends BankAccount implements Depositable, Withdrawable
```

... betyder, at **CheckingAccount** skal have både en **deposit**-metode og en **withdraw**-metode.

```
class SavingsAccount extends BankAccount implements InterestBearing, Depositable, Withdrawable {  
    private double interestRate;  
  
    public SavingsAccount(double balance, double interestRate) {  
        super(balance);  
        this.interestRate = interestRate;  
    }  
  
    // Implementing af interface InterestBearing  
    public void applyInterest() {  
  
    }  
    public void deposit(double amount) {  
        // ...  
    }  
    public void withdraw(double amount) {  
        // ...  
    }  
}
```

**BankAccount** kan vi nu simplificere til:

```
class BankAccount {  
    private double balance;  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

så kan alle konti arve fra **BankAccount** ...



og nu kan vi lave en liste med alle konti, uanset type:

```
ArrayList<BankAccount> accounts = new ArrayList<>();  
accounts.add(new CheckingAccount(owner1));  
accounts.add(new SavingsAccount(owner2, 0.05));  
accounts.add(new ChildrensAccount(owner3));
```

Og vi kan lave en **transfer**-metode, der kan

- hæve fra alle konti, der implementerer `Withdrawable`
- indsætte på alle konti, der implementerer `Depositable`

```
public void transfer(Withdrawable from, Depositable to, double amount) {  
    // ...  
}
```

Har i bemærket navngivningen af vores interfaces `InterestBearing`, `Depositable`, `Withdrawable`?

Hvad har de tilfælles?

InterestBearing, Depositable, Withdrawable

-able -endelsen indikerer, at det er noget en klasse *kan gøre* - ikke noget den *er*.

A photograph of three women sitting at a white table in a modern, bright office or study environment. They are all smiling and looking at their laptops. The woman on the left has blonde hair and is wearing a green top. The woman in the middle has dark hair and is wearing a purple sweater. The woman on the right has dark hair and is wearing a blue sweater. On the table are two laptops, a blue water bottle, and a blue mug. Above them, a large green plant hangs from a black frame. The background is a plain white wall.

# Interfaces vs. abstrakte klasser

Men kunne vi ikke gøre **BankAccount** til et interface i stedet?

```
interface BankAccount {  
    double getBalance();  
}
```

Ja og nej.

Mest nej, fordi **BankAccount** har

- tilstand (feltet `balance`) og
- fælles implementering (konstruktør og `getBalance`-metode deles af alle konti).
- ikke et -able navn

Adskillelse af interface og  
implementering



Interfaces adskiller **hvad** en klasse **kan gøre** fra **hvordan** den gør det.

```
public void transfer(Withdrawable from, Depositable to, double amount) {  
    // Hæv penge fra from-kontoen  
    // Indsæt penge på to-kontoen  
}
```

... behøver ikke at kende til de konkrete klasser, blot at de implementerer de rette interfaces.

Det kan være **CheckingAccount**, **SavingsAccount**, **ChildrensAccount** eller  
noget helt andet vi **ikke kender til endnu**!

Altså, vi kan tilføje nye kontotyper uden at ændre i **transfer**-metoden. Så længe de implementerer de rette interfaces, så virker det!

*"Programming to an interface, not an implementation".*

*- Klog mand*

# Løs kobling

Løs kobling mellem komponenter i vores system = mere fleksibelt design!

og de kloge folk der lavede Java SDK bruger samme trick:

```
public interface List<E> {  
    void add(E element);  
    E get(int index);  
    // ...  
}  
  
class ArrayList<E> implements List<E> {  
    // ...  
}
```

... er et interface, som **ArrayList**, **LinkedList** og andre klasser implementerer.

Så kan vi skrive metoder, der arbejder med `List<E>`, uden at kende til den konkrete implementering:

Demo: En **Bank**-klasse med **List<BankAccount>**



```
public class Bank {
    List<BankAccount> accounts;

    public Bank(List<BankAccount> accounts) {
        this.accounts = accounts;
    }

    public List<BankAccount> findAccountsAboveBalance(double threshold) {
        List<BankAccount> result = new ArrayList<>();
        for (BankAccount account : accounts) {
            if (account.getBalance() > threshold) {
                result.add(account);
            }
        }
        return result;
    }

    // ...
}
```

Nu kan vi oprette en **Bank** med en hvilken som helst **List**-implementering:

```
List<BankAccount> accounts = new LinkedList<>();  
Bank bank = new Bank(accounts);  
ArrayList<BankAccount> richAccounts = bank.findAccountsAboveBalance(1000);
```

eller

```
List<BankAccount> accounts = new ArrayList<>();  
Bank bank = new Bank(accounts);  
ArrayList<BankAccount> richAccounts = bank.findAccountsAboveBalance(1000);
```

eller din helt egen **Awesome03000List**-implementering af **List**:

```
class Awesome03000List<E> implements List<E> {  
    // ...  
    public void add(E element) {  
        // ...  
    }  
    public E get(int index) {  
        // ...  
    }  
    public int size() {  
        // ...  
    }  
    // ...  
}
```

```
List<BankAccount> accounts = new Awesome03000List<>();  
Bank bank = new Bank(accounts);  
ArrayList<BankAccount> richAccounts = bank.findAccountsAboveBalance(1000);
```